

テーマ：駐車場自動管理システム構成

1. はじめに

本レポートは、PBL (Project-Based Learning) 科目の一環として開発する駐車場管理システムの技術的設計について詳細に説明したものである。本システムでは、IoT (Internet of Things) 技術を活用し、駐車場の入退場プロセスを自動化するとともに、利用状況をリアルタイムで取得・表示することを目指す。

本レポートでは、以下の主要な課題について検討する。

- システムは何を実現すべきか (要求仕様)
- どのような構成で実現するか (アーキテクチャ設計)
- 必要となるハードウェアは何か (Hardware)
- どのようなソフトウェアを開発するか (Software)
- どのように試験し、改善するか (テスト計画)

機能要求

- 入退場検知：車両の入場・退場を自動的に検出する
- 車両識別：カメラによるナンバープレート認識 (OCR)
- 空き区画管理：区画ごとの駐車台数をリアルタイムに算出
- 情報表示：エリア別利用状況を入口端末で提示
- データ記録：入退場履歴をクラウドに保存
- 遠隔監視：管理者がリアルタイムで利用状況を確認

1 章：システム要求分析

1.1 現状の課題

従来の駐車場管理方式には、以下のような課題が存在する。

手作業による記録：係員が車両ナンバーを手書きで記録するため、入力ミスが発生しやすい。空き状況の把握遅れ：利用者は駐車場に近づくまで満車かどうかを確認できない。履歴の未保存：入庫・出庫時刻などの履歴が適切に保存されず、データが欠損する可能性がある。サービスの遅延：入口でチケットの受け取りや料金精算が必要となり、利用者の待ち時間が長くなる。

1.2 IoT 活用による解決効果

IoT 技術を導入することで、以下の利点が得られる。

自動認識の実現：カメラが車両を自動検知し、ナンバープレートを自動で読み取る。

リアルタイム情報提供：利用者は駐車場へ向かう前に、スマートフォンで空き状況を確認できる。履歴データの完全保存：クラウドに入退場履歴が長期的・一元的に保存される。迅速な入退場：車両が自動的に識別されるため、ゲートでの待ち時間が大幅に短縮される。

目的 (Purpose)

project の目的は、IoT 技術を活用したスマート駐車場管理システムを構築し、駐車場運用の効率化と利用者の利便性向上を実現することである。具体的な目標は以下のとおりである。

- 車両の入退場を自動的に識別する仕組みを実装すること
(カメラによるナンバープレート認識、センサー検知の自動化)
- 各エリアの空き状況をリアルタイムに算出・更新すること
(センサーデータを基にした占有状態の即時集計)
- 利用者および管理者に対し、最新の利用状況を表示する機能を提供すること
(入口表示器・Web ダッシュボード等)
- 入退場履歴および利用データをクラウドに保存し、長期的な分析を可能にすること
(データベースと分析基盤の整備)
- Server (cloud): 高い処理能力を持ち、大規模データの保存、詳細な解析、学習処理など高度な計算を担当する。

1.3 System Design

■ システム設計

本節では、IoT を活用した駐車場管理システムの全体アーキテクチャについて述べる。提案システムは、リアルタイムで入退場を検知し、車両番号の認識、空き状況の算出、クラウドへの記録、利用者アプリへの情報提供を自動化することを目的としている。図 1 に示すように、システムは以下の 4 つの主要コンポーネントで構成される。

■ Edge device

駐車区画やゲートに設置し、車両検知・画像取得・ゲートウェイへの送信を行う。

■ ゲートウェイ／処理ユニット (Processing)

監視ユニットから画像を受信し、OCR/ANPR によるナンバー認識や入退場の判定、空き状況の更新を実行し、クラウドへデータを送信する。

■ クラウド側 (Cloud-side)

入退場履歴の保存、空き状況の管理、通知処理などを行う。

■ ユーザーアプリケーション (User Application)

利用者が空き状況確認、予約、履歴閲覧などを行える。

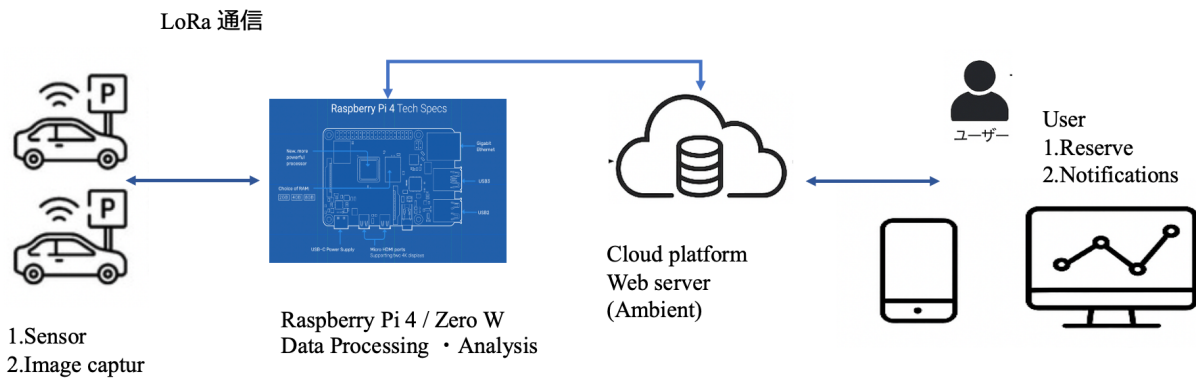


図1：メインシステムの基本アーキテクチャ

1.3 スマート駐車場管理システムは、エッジデバイス層 → ゲートウェイ層 → クラウドサーバー層の三層構造で設計されている。この構成により、低消費電力・長距離通信を実現しつつ、リアルタイムで駐車状況を把握できる堅牢なIoTアーキテクチャを実現した。

3-2. エッジデバイス層（Layer 1）

エッジデバイス層では、駐車場内の車両台数カウント、車両ナンバー認識、ゲート制御などの現場処理を担当する。センサー類はLoRa通信モジュールを使用し、低消費電力で長期間稼働できるよう設計されている。

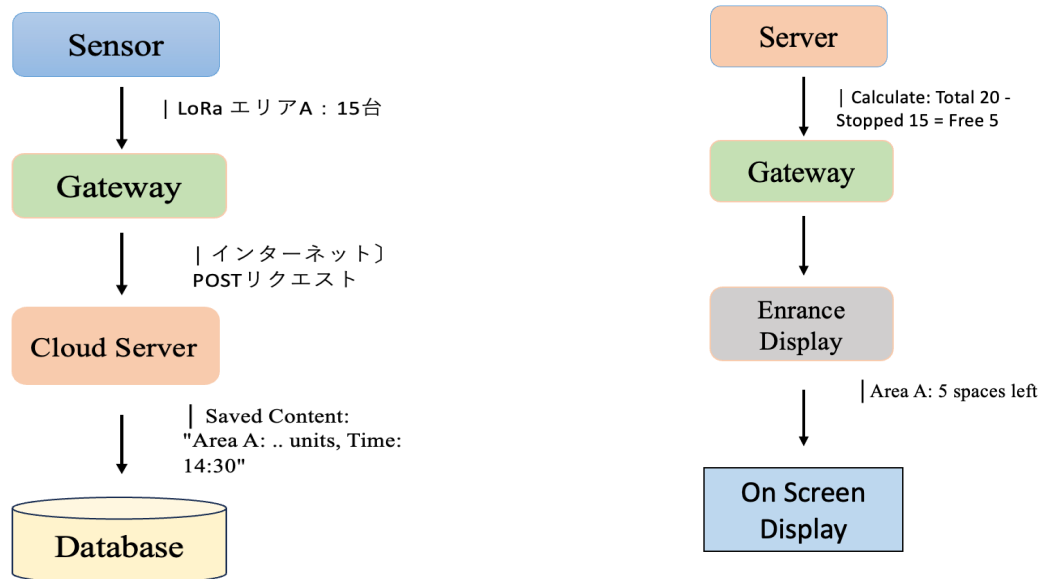
3-3. ゲートウェイ層（Layer 2）

ゲートウェイはLoRa信号を受信し、TCP/IPへ変換した上でクラウドにデータを送信する役割を持つ。

また、ネットワーク通信の監視や簡易的なローカル処理を行い、システム全体の安定性を確保する。

3-4. クラウドサーバー層（Layer 3）

クラウドサーバーでは収集されたデータの保存・分析を行い、Webダッシュボードやユーザー端末へリアルタイムデータを提供する。また、履歴レポート生成や料金計算などの高度な処理にも対応可能である。



Downstream

- 要素間の信号流れ（データフロー）
- エッジデバイス：車両検知 → 画像取得 → データ生成
- ゲートウェイ：LoRa 受信 → 整形 → LAN 経由でクラウドへ
- クラウド：車両番号解析 / 履歴登録
- ユーザーアプリ：空き情報参照 / 通知受信

Upstream

1.3 機能要件（何を実現すべきか）

A. 駐車車両検出機能

説明：駐車場エリアに設置したカメラにより、当該エリア内の駐車車両台数を自動的にカウントする。

例：エリア A：駐車可能台数 20 台 → カメラ検出：15 台駐車中

結果：空き 5 台 → サーバーへ送信

B. ナンバープレート認識機能

説明：駐車場の出入口にカメラを設置し、入出庫する車両のナンバープレートを認識する。

例：車両が接近 → カメラが撮影

画像：(12 A 3456) → OCR により読み取り

記録：車両 (12 A 3456) が 14:30 に入庫

3 層アーキテクチャを採用するのか

- Edge device：構造がシンプルで低コスト・低消費電力で動作する。主にセンシングと基本的な前処理のみを担当する。
- Gateway：複数のエッジデバイスからデータを収集し、インターネットやクラウドとの中継を行う役割を持つ。

2 章. 各モジュールの HW(Hardware) 実現

使用予定 HW	要点
ESP32 + カメラ (OV2640 等)	低消費電力動作、LoRa 送信機能
ESP32-CAM + 小型 LCD, or M5-stack	入口表示機能、安価な構成
Raspberry Pi 4	集約処理、Python 環境による通信制御
Cloud (Ambient + Firestore)	安定保存、REST API 対応

2.1 ESP32 モジュール

ESP32 とは？

ESP32 は Espressif Systems 社が開発した、低コストで高性能、Wi-Fi および Bluetooth 接続機能を備えたマイクロコントローラです。IoT (Internet of Things) やスマートデバイスに広く使用されています。



技術仕様

- プロセッサ: デュアルコア Tensilica LX6, 最大 240 MHz
- RAM: 約 520 KB, Flash: 最大 4 MB
- 通信: Wi-Fi 802.11 b/g/n, Bluetooth/BLE
- I/O ポート: GPIO, ADC, PWM, I2C, SPI, UART
- 消費電力: Deep-sleep モードで 10~150 μ A
- ペリフェラル対応: カメラ (OV2640)、LCD、各種センサー、LoRa モジュール。

ESP32 を選定した理由

低消費電力: Deep-sleep モードにより、バッテリー駆動の IoT システムで長時間動作可能。Wi-Fi および Bluetooth 統合: 1 つのチップで複数の通信方式に対応し、ネットワーク構成が簡単で安定。カメラ対応: ESP32-CAM モジュールは OV2640 カメラ搭載で、小型の画像・動画送信システムに最適。低コスト・小型・軽量: モジュールごとのコストが低く、設置が容易。プログラミングサポート: Arduino IDE や MicroPython で迅速に開発可能。LoRa や他ペリフェラルの統合: 通信範囲が限定された環境でも簡単に拡張可能。

2.2 LoRa モジュールとは？

LoRa モジュールは、低消費電力で長距離通信が可能な無線通信モジュールです。IoT デバイス間でのデータ送受信に広く使用され、Wi-Fi や Bluetooth よりも広い範囲で安定した通信を実現します。

項目	説明
通信方式	LoRa (Long Range), Chirp Spread Spectrum (CSS) 技術採用
周波数帯	433 / 868 / 915 MHz (地域による)
通信距離	2~15 km (環境による)
消費電力	送信時: 約 120 mA, 受信時: 約 10~15 mA, スリープ: 数 μ A
データ速度	0.3~50 kbps (長距離通信では低速)
特徴	長距離通信、干渉に強い、低消費電力、小型モジュール

3 章. 各モジュールのソフトウェア実現

1. Edge 側 (ESP32 などの端末) 使用言語 / 環境: MicroPython または Arduino

車両検知アルゴリズム

- センサー (モーションセンサーやカメラ) からのデータを処理し、車両の有無を判定する。

画像取得制御

- 必要に応じてカメラで画像を撮影し、後続処理や記録用に取得する。

LoRa 通信プロトコル

- 検知情報や画像メタデータを LoRa 無線通信でゲートウェイに送信する。

2. Gateway 側 (中継サーバ / Raspberry Pi 等)

使用言語: Python

LoRa 受信ドライバ: Edge デバイスから送られてくる LoRa 信号を受信する。

JSON 整形: 受信データをクラウドに送信可能な JSON 形式に変換する。

クラウド送信スクリプト: 整形済みデータを API 経由でクラウド (Firestore) に送信する。

3. Cloud 側 (Firestore / FCM)

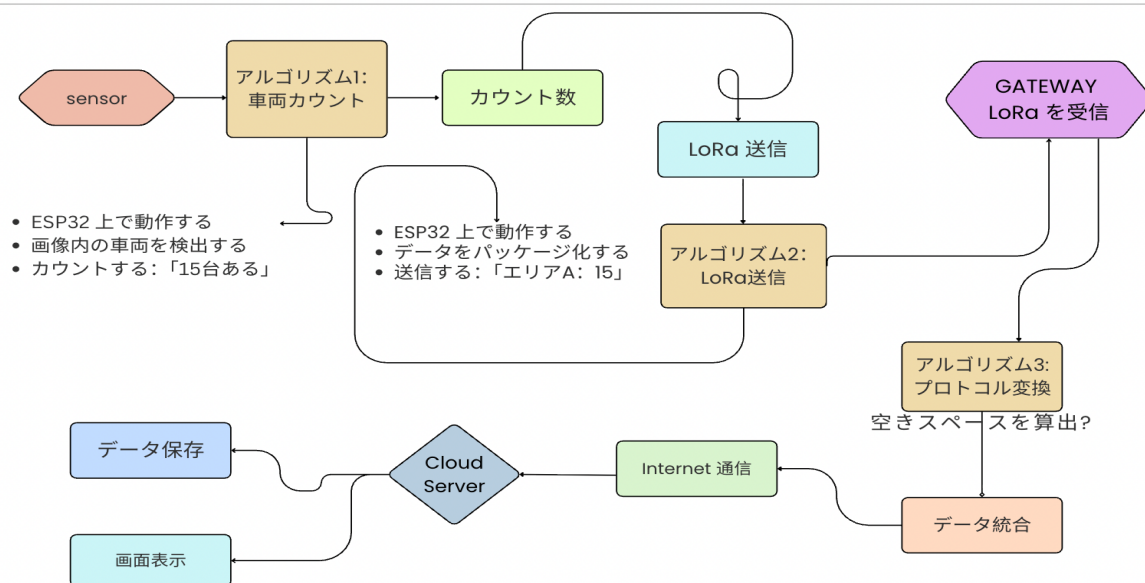
データスキーマ設計: 車両情報や通知履歴を保存するためのデータ構造を定義。

API 経由のデータ保存: Gateway から送信された JSON データを Firestore に保存する。

ユーザー通知処理。

3.1 アルゴリズムの動作

アルゴリズムはどこで、何のために使用されるのか



DATA FLOW

アルゴリズム	どこで動作する？	誰が実行する？	いつ動作する？
車両検出	ESP32 マイクロコントローラ (センサー上)	C / Python コード	画像を取得するたび
LoRa 送信	ESP32 (センサー上)	C コード	車両をカウントした後
プロトコル変換	Raspberry Pi (ゲートウェイ上)	Python コード	LoRa データが届いたとき
データ統合	Raspberry Pi (ゲートウェイ上)	Python コード	送信前
データ保存	☁ サーバー (クラウド上)	Python Flask	データが届いたとき
OCR ナンバー認識	Raspberry Pi (近接デバイス)	Python + Tesseract	車両が近づいたとき
データフィルタリング	ゲートウェイまたはサーバー	Python コード	データ処理時
省エネルギー制御	ESP32 (センサー上)	C コード	作業終了後

まとめ

このスマート駐車場システムの主要構成要素である。

(1) 車両検知アルゴリズム、(2) LoRa を用いたデータ通信、(3) データ統合および空きスペース算出、(4) ナンバー認識モジュールの全体構成が明確になりました。今後の開発において、最も時間とリソースを要すると考えられる部分は、アルゴリズムの精度向上と考えています。具体的には、背景差分方式による車両検知アルゴリズムの安定性向上、LoRa に送信するデータ量の最適化、空きスペースをリアルタイムで正確に算出する仕組みの確立など、技術的な課題が残っていると考えています。

References

- da Luz, Gustavo P. C. P., Gabriel Massuyoshi Sato, Luis Fernando Gomez Gonzalez, Juliana Freitag Borin. *Smart Parking with Pixel-Wise ROI Selection for Vehicle Detection Using YOLOv8, YOLOv9, YOLOv10, and YOLOv11*. arXiv, 2024.
- Al Amiri, Wesam; Baza, Mohamed; Banawan, Karim; Mahmoud, Mohamed; Alasmay, Waleed; Akkaya, Kemal. *Privacy-Preserving Smart Parking System Using Blockchain and Private Information Retrieval*. arXiv, 2019.
- Balfaqih, Mohammed; Jabbar, Waheb; Khayyat, Masha'el; Hassan, Rosilah. *Design and Development of Smart Parking System Based on Fog Computing and Internet of Things*. *Electronics*, **10** (24), 3184, 2021. DOI: 10.3390/electronics10243184.
- Napitupulu, Haposan Yoga Pradika; Nugraha, I Gde Dharma. *Fog Computing-Based System for Decentralized Smart Parking System by Using Firebase*. *Jurnal Nasional Teknik Elektro dan Teknologi Informasi*. DOI: 10.22146/jnteti.v13i1.10095.
- Smart Parking Services: An Overall Solution Based on Internet of Things and Fog Computing. *IEEE Access*. DOI 等未記載) .